AgroRiego — Arquitectura preliminar

Nov 1, 2026 · @Julian

Métodos Ágiles de Desarrollo · Dr. Carlos Jesús Hinojosa Rodríguez · Equipo 3

AgroRiego se plantea como un **monolito modular**: una sola API con módulos bien separados (autenticación, parcelas, dispositivos, riego y alertas), una app web instalable para el celular y una capa IoT por MQTT. Esta arquitectura es preliminar: se decide lo mínimo para el Sprint 1 y se revisa al cierre de cada sprint.

Principios de arquitectura continua

La arquitectura continua trata la arquitectura como algo que evoluciona con cada entrega, guiado por requisitos de calidad medibles y decisiones registradas. Así se aplican sus seis principios en el proyecto:

Principio	Cómo se aplica en AgroRiego
1. Arquitecturar productos, no solo proyectos	La arquitectura sigue al producto completo (las 16 historias), aunque en estos 3 sprints solo se construyen 9. Los módulos de consumo y reportes ya tienen lugar previsto.
2. Enfocarse en los atributos de calidad, no solo en funciones	Se definen requisitos medibles: la válvula responde en menos de 30 s, el panel se actualiza sin recargar, la app se usa en exteriores y con mala señal. Ver la última sección.
3. Retrasar las decisiones hasta que sean necesarias	La nube ya está elegida (AWS), pero se empieza con lo mínimo: una sola instancia EC2 con contenedores. Los servicios gestionados (Amazon RDS, AWS IoT Core), la separación en microservicios y el hardware real se deciden después; mientras, se usa un simulador de sensores.

Principio	Cómo se aplica en AgroRiego
4. Arquitecturar para el cambio: aprovechar el poder de lo pequeño	Módulos con fronteras claras y comunicación por eventos MQTT, para que cualquiera pueda separarse en un servicio propio sin reescribir los demás.
5. Arquitecturar para construir, probar, desplegar y operar	Integración continua en GitHub Actions, pruebas automatizadas por criterio de aceptación, despliegue en contenedores y monitoreo de los equipos en campo desde el Sprint 3.
6. Modelar la organización del equipo según el diseño del sistema	Cada módulo tiene un responsable natural: base de datos (Alejandro Erro), API (Daniel Valencia), nube e IoT (Julian Espinosa), app (Paul Seañez).

Las decisiones se registran como ADR (registros de decisión de arquitectura) en `docs/adr/` del repositorio, y se revisan en cada retrospectiva.

Diagrama de componentes

La API modular concentra la lógica; el campo solo habla con ella por MQTT

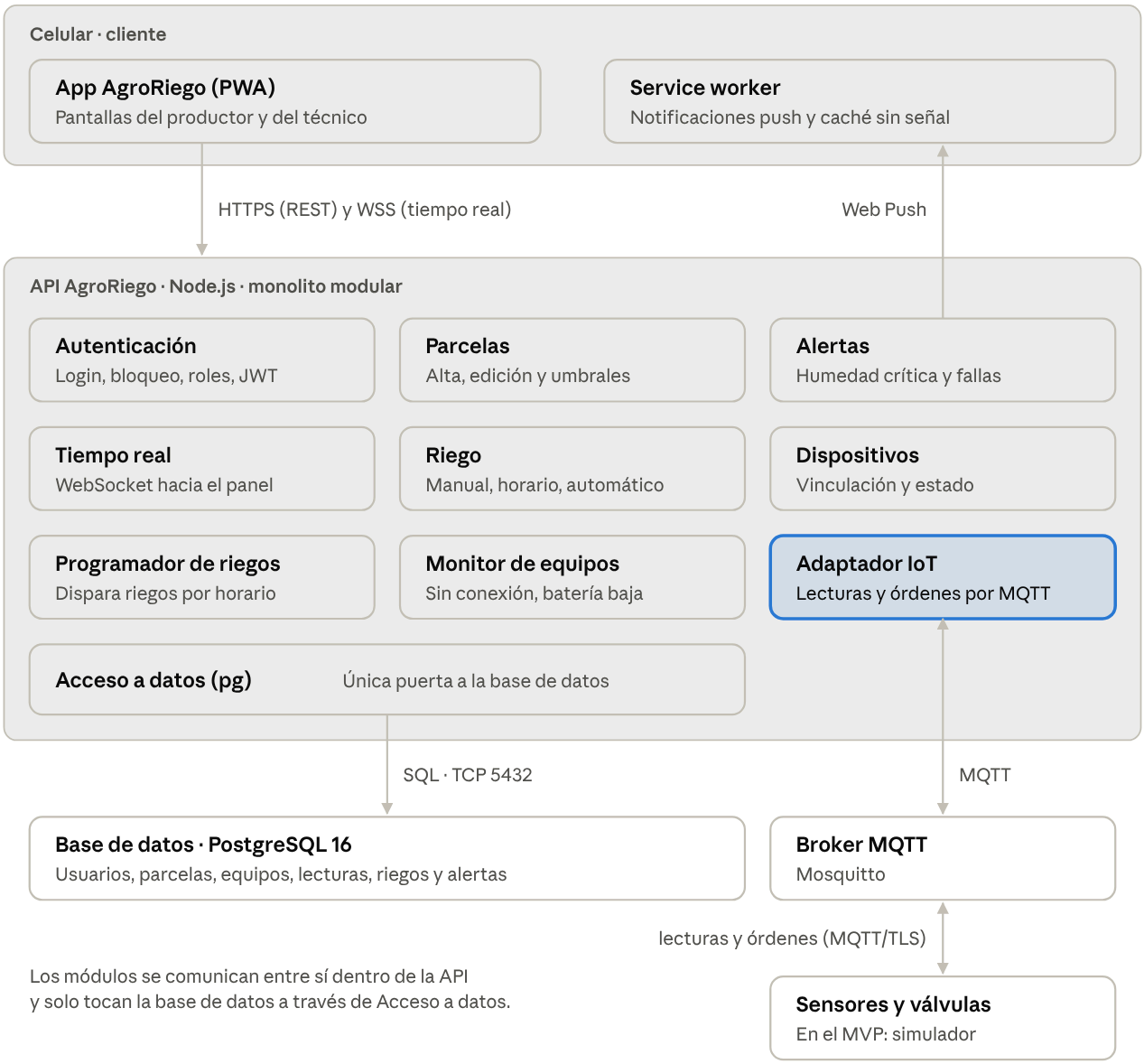


Diagrama de componentes · cliente, API con 10 módulos, datos e IoT

La app solo habla con la API. Los equipos de campo solo hablan con el broker MQTT, y el Adaptador IoT (resaltado) es el único módulo que traduce entre ese mundo y el resto del sistema.

Función de cada componente

Componente	Función	Se comunica con	Historias	Sprint
App AgroRiego (PWA)	Interfaz móvil del productor y del técnico: inicio de sesión, panel de parcelas, riego, alertas y alta de equipos. Se instala en el celular desde el navegador.	Módulos de la API por HTTPS (REST); servicio de tiempo real por WebSocket	Todas	1 a 3
Service worker	Parte de la PWA que recibe las notificaciones push aunque la app esté cerrada y guarda en caché la interfaz para abrirla con mala señal.	Módulo de alertas (Web Push)	HU-11, HU-12	2
Módulo de autenticación	Valida correo y contraseña, cuenta intentos fallidos, bloquea la cuenta 15 min, emite la sesión (JWT en cookie) y gestiona el restablecimiento de contraseña. Protege las rutas de los demás módulos según el rol.	Acceso a datos; servicio de correo	HU-01	1
Módulo de parcelas	Alta, consulta y edición de parcelas con sus umbrales de humedad. Garantiza que cada productor vea solo las suyas.	Acceso a datos	HU-03	1

Componente	Función	Se comunica con	Historias	Sprint
Módulo de dispositivos	Vincula sensores y válvulas a una parcela por código o QR, rechaza códigos duplicados y lleva el estado de cada equipo (esperando lectura, en línea, sin conexión).	Adaptador IoT; acceso a datos	HU-04	1
Adaptador IoT	Cliente MQTT de la API: recibe las lecturas de humedad, temperatura y batería, y publica las órdenes de abrir o cerrar válvulas. Aísla al resto del sistema del protocolo del hardware.	Broker MQTT; módulos de dispositivos, riego y alertas	HU-04, HU-05, HU-07	1 y 2
Servicio de tiempo real	Envía a la app cada lectura nueva y cada cambio de estado de riego, para que el panel se actualice sin recargar.	App (WebSocket); adaptador IoT	HU-05, HU-07	2
Módulo de riego	Abre y cierra la válvula a petición del productor, por horario o automáticamente por umbral de humedad, y registra cada riego con su duración.	Adaptador IoT; programador; acceso a datos	HU-07, HU-08, HU-09	2 y 3
Programador de riegos	Revisa cada minuto los programas por horario y dispara el inicio y fin de cada riego programado.	Módulo de riego	HU-08	3

Componente	Función	Se comunica con	Historias	Sprint
Módulo de alertas	Detecta humedad crítica, evita repetir la misma alerta en 1 h y envía las notificaciones push.	Adaptador IoT; monitor de equipos; service worker	HU-11, HU-12	2 y 3
Monitor de equipos	Marca «Sin conexión» a los equipos que llevan 30 min sin reportar y genera avisos de batería baja para el técnico.	Módulo de dispositivos; módulo de alertas	HU-12	3
Acceso a datos (node-postgres)	Conexión única a la base de datos (pool de node-postgres) y migraciones SQL versionadas; cada módulo tiene sus consultas en su propio archivo y ninguno toca las tablas de otro. Facilita cambiar o separar la base de datos más adelante.	Base de datos	Todas	1
Base de datos (PostgreSQL)	Guarda usuarios, parcelas, dispositivos, lecturas, riegos, programas y alertas.	Acceso a datos	Todas	1
Broker MQTT (Mosquitto)	Intermediario de mensajes entre los equipos de campo y la API. Los equipos publican lecturas y se suscriben a sus órdenes.	Adaptador IoT; sensores y válvulas	HU-04, HU-05, HU-07	1

Componente	Función	Se comunica con	Historias	Sprint
Sensores y válvulas	En campo: el sensor mide humedad y temperatura del suelo cada 5 min; la válvula abre o cierra el agua. En el MVP se reemplazan por el **simulador**, que publica los mismos mensajes.	Broker MQTT	HU-04, HU-05, HU-07	1

Servicios externos: un proveedor de correo para los enlaces de restablecimiento (en el MVP se muestran en la consola) y el servicio de notificaciones push del navegador.

Diagrama de despliegue

Todo el backend corre en contenedores en la nube; los datos quedan en red privada

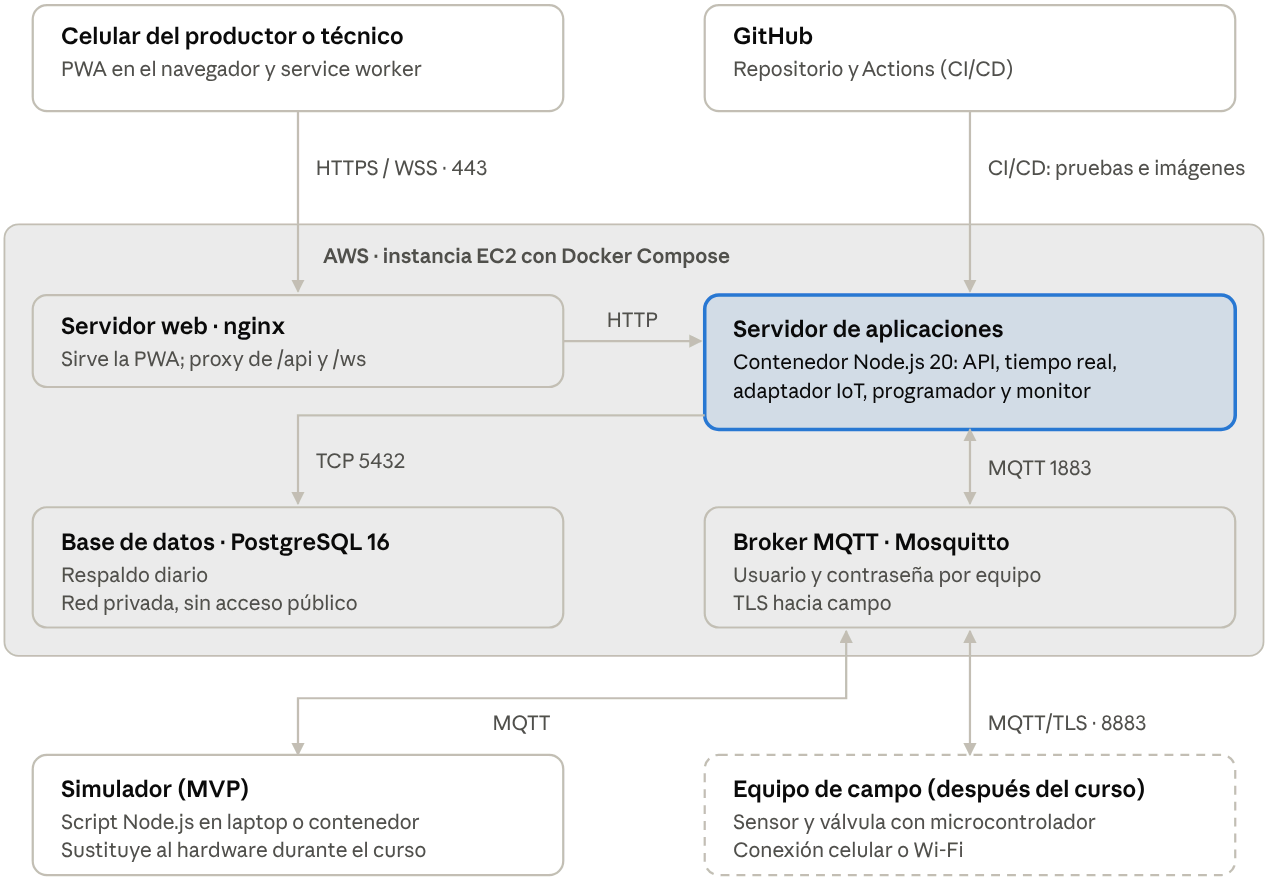


Diagrama de despliegue · 8 nodos en celular, nube, GitHub y campo

El celular solo entra por el servidor web en el puerto 443. La base de datos y el broker quedan en la red privada de la nube; desde campo solo se expone el puerto MQTT con TLS.

Función de cada nodo

Nodo	Qué se despliega ahí	Función	Comunicación
Celular del productor o técnico	Navegador móvil con la PWA instalada y su service worker	Ejecuta la interfaz y recibe las notificaciones push	HTTPS y WSS (puerto 443) hacia la nube

Nodo	Qué se despliega ahí	Función	Comunicación
Servidor web	Contenedor nginx con los archivos estáticos de la PWA	Entrega la app al celular y redirige `/api` y `/ws` al servidor de aplicaciones	HTTPS 443 desde internet; HTTP interno hacia la API
Servidor de aplicaciones	Contenedor Docker con Node.js 20: la API con todos sus módulos, el servicio de tiempo real, el adaptador IoT, el programador y el monitor	Ejecuta toda la lógica de negocio	HTTP/WS interno; TCP 5432 a la base de datos; MQTT 1883 al broker dentro de la red privada
Servidor de base de datos	PostgreSQL 16 con respaldo diario	Almacena todos los datos; no es accesible desde internet	TCP 5432, solo desde el servidor de aplicaciones
Broker MQTT	Contenedor Mosquitto con usuario y contraseña por equipo	Recibe lecturas y distribuye órdenes de riego	MQTT sobre TLS 8883 desde campo; 1883 interno
Equipo de campo	Sensor de humedad y válvula con un microcontrolador con conexión celular o Wi-Fi	Mide la humedad cada 5 min y abre o cierra el agua	MQTT/TLS 8883 hacia el broker
Simulador (MVP)	Script Node.js en una laptop o contenedor	Sustituye al equipo de campo mientras no hay hardware; publica lecturas y responde a las órdenes	MQTT hacia el broker
GitHub (repositorio y Actions)	Código fuente y pipeline de integración continua	En cada pull request corre las pruebas; al integrar a `main` construye las imágenes y despliega al ambiente de pruebas	HTTPS hacia la nube

En el Sprint 1 todos los nodos de nube corren con Docker Compose en una sola instancia Amazon EC2, pagada con los créditos de AWS del equipo y con una alerta de AWS Budgets para no rebasarlos. Como cada uno es un contenedor separado, después pueden moverse a servicios gestionados (Amazon RDS para PostgreSQL y AWS IoT Core para MQTT) sin cambiar el código.

Decisiones de arquitectura y requisitos de calidad

Requisitos de calidad

Atributo	Escenario medible	Cómo lo atiende la arquitectura
Rendimiento	La válvula se abre en menos de 30 s después de confirmar el riego	Órdenes por MQTT y confirmación de estado desde el equipo
Oportunidad de datos	Una lectura nueva aparece en el panel en menos de 10 s sin recargar	Servicio de tiempo real por WebSocket
Disponibilidad	Si un equipo deja de reportar 30 min, el técnico recibe aviso	Monitor de equipos y alertas push
Seguridad	Solo el dueño ve y controla sus parcelas; cuenta bloqueada tras 5 intentos	Módulo de autenticación con roles; base de datos sin acceso público; MQTT con TLS
Usabilidad en campo	Se usa en exteriores con un celular y señal irregular	PWA con caché, alto contraste y botones de 44 px o más
Modificabilidad	Un módulo puede separarse en servicio propio sin reescribir los demás	Monolito modular, acceso a datos centralizado y eventos MQTT

Decisiones registradas (ADR)

ADR	Decisión	Motivo	Se revisa en
ADR-001	Monolito modular en lugar de microservicios	Equipo de 5 personas y 6 semanas; menos complejidad de despliegue	Retrospectiva del Sprint 3
ADR-002	MQTT para comunicar los equipos de campo	Protocolo ligero, pensado para conexiones inestables y equipos de bajo consumo	Cuando se elija el hardware real
ADR-003	PWA en lugar de app nativa	Una sola base de código para Android y iPhone, sin tiendas de apps	Al terminar el Sprint 2
ADR-004	AWS: una instancia EC2 con Docker Compose	Cabe en los créditos de AWS del equipo; por usar contenedores, se puede pasar a RDS o IoT Core sin cambiar el código	Si el uso se acerca al límite de créditos
ADR-005	Simulador en lugar de hardware en el MVP	No se cuenta con sensores ni válvulas durante el curso	Después del curso

Cambio registrado durante el Sprint 1 (arquitectura continua: la decisión se ajustó al construir):

ADR	Decisión	Motivo	Se revisa en
ADR-006	JavaScript (módulos ES) y SQL con node-postgres en lugar de TypeScript y Prisma	Menos pasos de compilación y herramientas para un equipo nuevo en el stack; las migraciones quedan como SQL legible y revisable en cada pull request	Retrospectiva del Sprint 2